

Tipos de datos

Java es un lenguaje de tipado fuerte, lo que significa que tienes que decirle al compilador exactamente qué tipo de información vas a guardar en cada "caja" antes de usarla.

1. `int`: Guarda números enteros sin decimales.

```
// Archivo: PruebaEnteros.java
public class PruebaEnteros {
    public static void main(String[] args) {
        // Declaramos obligatoriamente que es un 'int'
        int usuariosActivos = 45;
        usuariosActivos++; // El incremento de Java que tanto me gusta

        // Así imprimimos en la consola de Java:
        System.out.println("Usuarios en la terminal: " +
usuariosActivos);
    }
}
```

2. `double`: Almacena números con punto decimal de alta precisión.

```
// Archivo: PruebaDecimales.java
public class PruebaDecimales {
    public static void main(String[] args) {
        double precioProducto = 99.99;
        double descuento = 0.15;
        double total = precioProducto * (1.0 - descuento);

        System.out.println("Total a pagar: " + total);
    }
}
```

3. `boolean`: Solo admite `true` o `false` en minúsculas. Nada de números, nada de texto.

```
// Archivo: PruebaBooleanos.java
public class PruebaBooleanos {
    public static void main(String[] args) {
        boolean tieneAcceso = true; // ¡Estrictamente en minúscula!

        if (tieneAcceso) { // La condición va OBLIGATORIAMENTE entre
paréntesis
            System.out.println("Acceso concedido desde la terminal de
Java");
        }
    }
}
```

4. String: Un String en Java es en realidad un objeto que envuelve una secuencia de caracteres

```
// Archivo: PruebaStrings.java
public class PruebaStrings {
    public static void main(String[] args) {

        // Declaramos los objetos String (¡S mayúscula!)
        String usuarioGuardado = "admin";
    }
}
```

5. Integer: En Java las listas son serias y estructuradas, se llaman ArrayList y solo aceptan objetos. Por eso integer sustituye a int en las listas.

```
// Archivo: PruebaInteger.java
import java.util.ArrayList; // Necesitamos importar la herramienta de
listas

public class PruebaInteger {
    public static void main(String[] args) {
        // ERROR DE EXAMEN: ArrayList<int> lista = new ArrayList<>();
        <-- Esto NO COMPILA.

        // FORMA CORRECTA: Usamos la clase wrapper Integer
        ArrayList<Integer> notasExamen = new ArrayList<>();

        // Añadimos datos. Java convierte automáticamente el 'int' a
        'Integer' por detrás (Autoboxing)
        notasExamen.add(8);
        notasExamen.add(5);
        notasExamen.add(10);

        System.out.println("Notas guardadas en la lista: " +
        notasExamen);

        // Otro uso vital de Integer: Convertir un texto de la terminal
        a número para poder operar
        String textoDesdeTerminal = "25";
        int edad = Integer.parseInt(textoDesdeTerminal); // Convierte el
        String "25" en el int 25
        System.out.println("El año que viene tendrás: " + (edad + 1));
    }
}
```

Prioridades de operadores:

Java los bloques están muy claros. Aquí tenéis la tabla simplificada de prioridades de mayor a menor importancia:

1. Postfijos/Prefijos: ++, -- (¡Esto en Python ni existe! Incrementa o decrementa una variable directamente).
2. Multiplicativos: *, /, % (Multiplicación, división y el resto de la división).
3. Aditivos: +, - (Suma y resta).
4. Relacionales: <, >, <=, >=.
5. Igualdad: ==, !=.
6. Lógicos: && (Equivalente al and de Python) y luego || (Equivalente al or de Python).
7. Asignación: =, +=, -=, etc. (Los últimos de la fila).
8. Exponente (Potencias):

```
double base = 2;
double exponente = 3;
double resultado = Math.pow(base, exponente); // Resultado: 8.0

// Si necesitas el resultado en un entero (int), hay que hacer un
"casting":
int resultadoEntero = (int) Math.pow(base, exponente);
```

Sentencias condicionales:

Java usamos paréntesis obligatorios para las condiciones y llaves {} para abrir y cerrar los bloques de código.

1. La estructura if - else (y else if):

Cuándo se usa: Cuando tienes dos o más caminos posibles. Si la condición es verdadera se hace una cosa, y si es falsa (else), se hace otra.

```
// Archivo: CondicionalElse.java
public class CondicionalElse {
    public static void main(String[] args) {
        int nota = 4;

        if (nota >= 7) {
            System.out.println("Notable");
        } else if (nota >= 5) { // Equivalente al 'elif'
            System.out.println("Aprobado justo");
        } else {
            System.out.println("Suspenso");
        }
    }
}
```

2. La sentencia switch:

Cuando tienes una sola variable y quieres evaluar múltiples casos posibles de forma ordenada.

```
// Archivo: PruebaSwitch.java
public class PruebaSwitch {
    public static void main(String[] args) {
        int mes = 2;

        switch (mes) {
            case 1:
                System.out.println("Enero");
                break; // ¡VITAL! Si no pones 'break', el código sigue
ejecutando los siguientes casos.
            case 2:
                System.out.println("Febrero");
                break;
            case 3:
                System.out.println("Marzo");
                break;
            default: // Es el equivalente al 'else' final. Si no es ni
1, ni 2, ni 3...
                System.out.println("Otro mes");
        }
    }
}
```

```
        break;
    }
}
```

Matrices:

En Java, una matriz (o array) es un contenedor indexado que guarda un conjunto de elementos del mismo y único tipo de dato. Además, tienen un tamaño fijo: una vez que lo creas, no se puede encoger ni estirar.

Java (Array rígido y seguro): `int[] misNumeros;` (Solo guardará enteros).

¿Cómo se crean (Declaración e Inicialización)?

1. Conociendo el tamaño pero no los valores iniciales

```
int[] notas = new int[5]; // Array listo para guardar 5 notas.
```

2. Conociendo los valores iniciales inmediatamente

```
int[] notas = {10, 8, 9};
```

¿Cómo añadir elementos a las matrices?

Aquí viene el gran choque cultural para un programador de Python: ¡En los arrays de Java NO existe el método `.append()`! No puedes meter más elementos de los que definiste al principio porque el tamaño en memoria es estático.

```
int[] numeros = new int[3];
numeros[0] = 14; // Añades en la primera posición
numeros[1] = 25; // Añades en la segunda posición
numeros[2] = 8;  // Añades en la terceraposición
// numeros[3] = 99; --> ¡ERROR! Lanza una excepción en tiempo de
ejecución
```

¿Cómo moverse (recorrer) y acceder a los elementos?

Al igual que en Python, los índices empiezan siempre en 0. Para ver cuántos elementos tiene, en Python usabas la función `len(lista)`. En Java, usamos la propiedad `.length`.

Acceso individual:

```
System.out.println(notas[0])
```

Bucleo para recorrerlo (Tradicional):

```
for (int i = 0; i < notas.length; i++) {  
    System.out.println(notas[i]);  
}
```

Matrices Bidimensionales (Tablas)

Una matriz bidimensional es simplemente un "array de arrays". Piensa en ella como una cuadrícula o tabla con filas y columnas.

Crearla vacía definiendo el tamaño (Filas x Columnas)

```
int[][] miTabla = new int[3][4];
```

Crearla inicializando los datos de golpe

```
int[][] matriz = {  
    {10, 20, 30},  
    {40, 50, 60}  
}; // ¡No te olvides del punto y coma final!
```

¿Cómo moverse (recorrer) en las matrices bidimensionales?

Para movernos por la cuadrícula necesitamos dos bucles for anidados: el primero controlará la fila en la que estamos parados y el segundo recorrerá las columnas de esa fila.

Para saber el tamaño, usamos la propiedad `.length`:

- `matriz.length` nos da el número total de filas.
- `matriz[i].length` nos da el número de columnas de esa fila en específico.

Código para recorrer e imprimir la matriz:

```
for (int i = 0; i < matriz.length; i++) {           // Recorre filas
    for (int j = 0; j < matriz[i].length; j++) { // Recorre columnas
        System.out.print(matriz[i][j] + " ");
    }
    System.out.println(); // Salto de línea al terminar cada fila
}
```

¿Cómo añadir o modificar elementos a Matrices Bidimensionales?

Al igual que con las matrices simples, el tamaño de una matriz bidimensional en Java es fijo. No existe un `.append()` para meter una fila nueva de la nada. Lo que hacemos es asignar valores directamente a las coordenadas existentes.

```
matriz[0][1] = 99;
```

Bucle for-each:

¡Excelente! El bucle for-each (que en Java formalmente se conoce como el bucle for mejorado o enhanced for) es lo más parecido que vas a encontrar al cómodo **for elemento in lista**: de Python. Dentro del paréntesis declaras una variable del mismo tipo de dato que almacena el array (en este caso `int nota`), pones dos puntos (`:`) que significan "dentro de", y luego el nombre de tu array (`notas`).

Matrices Unidimensionales:

Imagina que tenemos un array con notas de exámenes. Queremos leer cada nota.

```
int[] notas = {10, 8, 9};

for (int nota : notas) {
    System.out.println(nota);
}
```

Matrices Bidimensionales:

```
int[][] matriz = {
    {1, 2},
    {3, 4}
};
```

```
// 1. El primer bucle extrae cada FILA (que es un array int[])
for (int[] fila : matriz) {

    // 2. El segundo bucle extrae cada ELEMENTO (int) de esa fila
    for (int elemento : fila) {
        System.out.print(elemento + " ");
    }
    System.out.println(); // Salto de línea al terminar la fila
}
```

Metodos estaticos Arrays

Ordenar Arrays:

Java nos proporciona una clase de utilidad llamada `java.util.Arrays` que viene con métodos estáticos listos para usar. Uno de ellos es `sort()`.

```
import java.util.Arrays;
public class Listas {
    public static void main(String[] args) {
        int[] edade = {25, 27, 28, 22, 24, 29};
        // Ordena la tabla
        Listas.sort(edade);
        // Mostrar resultado
        for (int e : edade) {
            System.out.println(e);
        }
    }
}
```


Buscar sin binarySearch en Arrays:

Esta es la forma tradicional. Consiste en abrir un bucle e ir mirando uno a uno todos los elementos del array desde el principio hasta el final hasta encontrar el que buscas. Es el único método que funciona si el array está desordenado.

```
//siempre usar import java.util.Arrays; para los emtodos estaticos
import java.util.Arrays;
public class Listas {
    public static void main(String[] args) {
        int[] edade = {25, 27, 28, 22, 24, 29};
        // Ordena la tabla
        Arrays.sort(edade);
        // Mostrar resultado
        //Agora edade = {22, 24, 25, 27, 28, 29};
        int encontrar= 28;
        int i =0;
        boolean encontrado = false;
        for (;i<edade.length; i++) {
            if (edade[i]== encontrar){
                encontrado = true;
                break;
            }
        }
        if (encontrado){
            System.out.println("Esta en posición: " + i);
        }
        else{
            System.out.println("Non foi encontrado: -1");
        }
    }
}
```

Buscar con `binarySearch` en Arrays:

La búsqueda binaria es infinitamente más rápida que la secuencial, pero tiene una regla de oro obligatoria: el array TIENE QUE ESTAR ORDENADO previamente. Si no lo está, te devolverá basura.

Funciona dividiendo el array por la mitad repetidamente (como cuando buscas una palabra en un diccionario físico). Java ya tiene este algoritmo programado dentro de la clase de utilidad `java.util.Arrays`.

```
//siempre usar import java.util.Arrays; para los emtodos estaticos
import java.util.Arrays;
public class Listas {
    public static void main(String[] args) {
        int[] edade = {25, 27, 28, 22, 24, 29};
        // Ordena la tabla
        Arrays.sort(edade);
        // Mostrar resultado
        //Agora edade = {22, 24, 25, 27, 28, 29};

        int pos = Arrays.binarySearch(edade,28);
        if (pos>=0){
            System.out.println("Encontrado en posición: " + pos);
        }
        else{
            System.out.println("No encontrado");
        }
    }
}
```

Copiar SIN método estático :

Esta es la forma artesanal y la que demuestra que entiendes qué pasa por dentro. Consiste en crear un nuevo array vacío del mismo tamaño que el original, y luego abrir un bucle for para ir clonando los valores casilla por casilla.

```
//creamos un array del mismo tamaño
int edade2[] = new int[edade.length];
//recordemos la primera lista para pasar la posición a la nueva lista
for (int j=0; j<edade.length; j++){
    edade2[j] = edade[j];
}
```

Copiar CON método estático:

Cuando necesitas rendimiento y escribir menos código, Java pone a tu disposición un método estático súper optimizado dentro de la clase del sistema llamado `System.arraycopy()`.

```
//Copia toda la lista
int edade3[] = Arrays.copyOf(edade, edade.length);
//Copia por rango
int edade4[] = Arrays.copyOfRange(edade, 1, 4);
```

Comparar contenido:

Para no tener que escribir esa función en cada proyecto, Java nos ofrece el método estático `Arrays.equals()`. Este método ya tiene programada exactamente la misma lógica que acabamos de hacer a mano: compara tamaños y luego contenidos de forma secuencial.

```
import java.util.Arrays; // ¡Que no se te olvide importar la
herramienta!
int[] array1 = {1, 2, 3};
int[] array2 = {1, 2, 3};
int[] array3 = {1, 2, 4};
boolean caso1 = Arrays.equals(array1, array2); // Devolverá true
boolean caso2 = Arrays.equals(array1, array3); // Devolverá false (el
último elemento cambia)
```

EJEMPLO COMPLEJO REAL:

Crear a función que reciba como parámetro unha táboa unidimensional e devolte outra táboa cos valores da primeira, pero na que non haxa repeticións nos valores.

```
//siempre usar import java.util.Arrays; para los emtodos estaticos
import java.util.Arrays;
public class Listas {
    public static void main(String[] args) {
        //Creo la tabla
        int [] numeros ={ 1,2,4,9,8,2,4,7,2,5};
        //Ordeno
        Arrays.sort(numeros);
        //Creo el array vacio;
        int numeros2[] = new int[numeros.length];
        int posicion = 0;
        // Guardo el primer numero
        numeros2[posicion] = numeros[0];
        //Recorro la primera lista desde la posicion 1
        for (int i=1; i< numeros.length;i++){
            //comparo si un numero es distinto a su anterior;
            if (numeros[i] != numeros[i - 1]){
                //añado una posición a la lista 2
                posicion++;
                // Lo añadido a la lista
                numeros2[posicion]=numeros[i];
            }
        }
        //Copia para quitar los 0
        int numeros3[]= Arrays.copyOfRange(numeros2,0,7);
        //Recorrer la tabla nueva sin los 0
        for (int j=0; j<numeros3.length;j++){
            System.out.println(numeros3[j]);
        }
    }
}
```

String:

Un String es una secuencia de caracteres (letras, números, espacios en blanco y caracteres especiales como ?, @, etc.). Por ejemplo, "Hola Mundo" o "user123" son cadenas de texto. La gran diferencia con Python es que en Java los Strings no son un tipo de dato primitivo (como sí lo son int, double o boolean). Los string son inmutables

Declaración:

1. Asignación en la misma línea

```
String nombre = "Alejandro";  
String saludo = "¡Hola, mundo!";  
String vacio = ""; // Una cadena válida, pero vacía
```

2. Declarar primero y asignar el valor después

A veces no sabes qué texto va a contener la variable desde el principio (por ejemplo, porque estás esperando a que el usuario lo escriba por teclado). En ese caso, puedes separarlo en dos pasos:

```
// 1. Declaras la variable (se crea el espacio en la memoria)  
String mensaje;  
  
// 2. Le asignas el valor más adelante en tu código  
mensaje = "Bienvenidos a la clase de Java";
```

Longitud de una cadena:

Cuando estás trabajando con cadenas de texto (String), length() es un método (una función interna de la clase). Por lo tanto, es obligatorio ponerle paréntesis al final (). Devuelve el número total de caracteres que tiene la cadena (contando letras, números, símbolos y espacios en blanco).

```
String texto = "Hola Mundo";  
int tamanoTexto = texto.length(); // Devuelve 10 (lleva paréntesis)  
System.out.println(tamanoTexto);
```

Concatenación:

1. El operador + (El más utilizado)

Es la forma directa y la que heredamos mentalmente de Python. Puedes unir variables String, literales de texto e incluso mezclar texto con números u otros tipos de datos (Java convertirá automáticamente los números a texto por detrás).

```
String nombre = "Ana";
String apellido = "García";

// Concatenación simple usando el operador +
String nombreCompleto = nombre + " " + apellido;
System.out.println(nombreCompleto); // Imprime: Ana García
```

2. El método de instancia .concat()

Es un método que pertenece a cada objeto String. A diferencia de un método estático (que se llama usando el nombre de la clase), este se invoca directamente desde una variable de texto. Su única limitación es que solo acepta otro String como parámetro.

```
String saludo = "Hola ";
String destino = "Mundo";

// Invocamos el método desde la variable 'saludo'
String resultado = saludo.concat(destino);
System.out.println(resultado); // Imprime: Hola Mundo
```

Concatenación de datos no String:

Si usas el operador + y al menos uno de los operandos es un String, Java convertirá automáticamente el otro dato a texto antes de pegarlos.

```
public static void main(String args[]) {
    String novaCadea = "Aprendendo Java" + 17;
    System.out.println(novaCadea); //Aprendendo Java 17
    System.out.println("Total : " + 17 + 17); //Total: 1717
    System.out.println("Total : " + (17 + 17)); //Total: 34
    String numCadea = "17" + 17;
    System.out.println(numCadea); //1717
} //Fin método main
```

Acceso a cada caracteres:

Sin embargo, a diferencia de Python o de los propios arrays de Java, no puedes usar los corchetes [] para acceder a una letra; si escribes texto[0], el compilador dará un error inmediato.

Extraer un carácter específico: El método .charAt(índice)

Para recuperar el carácter que se encuentra en una posición determinada, debes utilizar obligatoriamente el método .charAt(int index). Al igual que en los arrays, los índices en un String son de base cero; es decir, el primer carácter está en la posición 0 y el último se encuentra en la posición longitud - 1.

```
String mensaje = "Java";

// Guardamos el primer carácter (índice 0) en una variable de tipo char
char primeraLetra = mensaje.charAt(0); // Devuelve 'J'
char terceraLetra = mensaje.charAt(2); // Devuelve 'v'

System.out.println("La primera letra es: " + primeraLetra);
```

Recorrer todos los caracteres (Bucle secuencial)